

2520030340

OS PRACTICAL – 6

PART – A: CLIENT-SERVER COMMUNICATION USING NAMED PIPES

AIM

To create a client-server application using **Named Pipes (FIFOs)**, where the client sends messages to the server and the server processes and responds to them. Also, to analyze FIFO behavior when multiple clients communicate simultaneously.

OBJECTIVES

1. To understand Inter-Process Communication using Named Pipes.
 2. To create and use a FIFO using `mkfifo()`.
 3. To implement communication between client and server processes.
 4. To send messages from the client to the server.
 5. To send responses from the server to the client.
 6. To demonstrate communication with multiple clients.
-

REQUIREMENTS

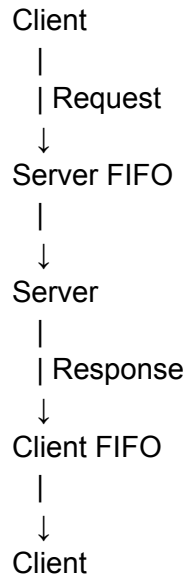
- Ubuntu/Linux operating system
 - GCC compiler
 - C programming language
 - Named Pipes (FIFO)
 - `mkfifo()`
 - `open()`
 - `read()`
 - `write()`
 - `fork()`
 - `close()`
 - `unlink()`
-

THEORY

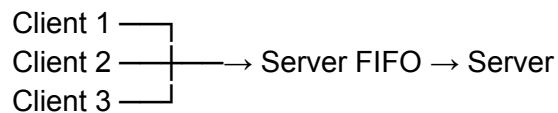
A **FIFO (First In First Out)** is a type of named pipe used for communication between processes.

Unlike an anonymous pipe, a FIFO has a name in the file system and can be used by unrelated processes.

In this program:



For multiple clients:



The server creates a separate child process to handle each client request.

ALGORITHM

1. Start the server program.
2. Create a server FIFO using `mkfifo()`.
3. Open the server FIFO for reading.
4. Wait for a client request.
5. Start the client program.
6. Create a separate FIFO for the client.
7. Client sends its PID and message to the server FIFO.

8. Server reads the client request.
 9. Server creates a child process using `fork()`.
 10. The child opens the client's FIFO.
 11. The server sends a response to the client.
 12. Client reads and displays the response.
 13. Multiple clients can communicate with the server.
 14. Stop the program.
-

PROGRAM 1: SERVER

File Name

fifo_server.c

COPY-PASTE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>

#define SERVER_FIFO "/tmp/os_server_fifo"

typedef struct
{
    pid_t client_pid;
    char message[256];
} Request;

int main()
{
    int server_fd;

    printf("===== FIFO SERVER =====\n");

    /* Create server FIFO */
    mkfifo(SERVER_FIFO, 0666);

    /* Open server FIFO */
```

```

server_fd = open(SERVER_FIFO, O_RDONLY);

if (server_fd == -1)
{
    perror("open");
    return 1;
}

printf("Server is waiting for clients...\n");

while (1)
{
    Request req;

    ssize_t bytes = read(server_fd, &req, sizeof(req));

    if (bytes <= 0)
        continue;

    printf("\nClient %d connected\n", req.client_pid);
    printf("Message: %s\n", req.message);

    pid_t child = fork();

    if (child == 0)
    {
        char client_fifo[100];

        snprintf(client_fifo,
                 sizeof(client_fifo),
                 "/tmp/client_%d_fifo",
                 req.client_pid);

        int client_fd = open(client_fifo, O_WRONLY);

        if (client_fd == -1)
        {
            perror("client fifo");
            exit(1);
        }

        char response[256];

        snprintf(response,

```

```
        sizeof(response),
        "Server received your message: %s",
        req.message);

    write(client_fd, response, strlen(response) + 1);

    close(client_fd);

    exit(0);
}
}

close(server_fd);

return 0;
}
```

PROGRAM 2: CLIENT

File Name

fifo_client.c

COPY-PASTE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>

#define SERVER_FIFO "/tmp/os_server_fifo"

typedef struct
{
    pid_t client_pid;
    char message[256];
} Request;

int main()
{
```

```

pid_t pid = getpid();

char client_fifo[100];

snprintf(client_fifo,
         sizeof(client_fifo),
         "/tmp/client_%d_fifo",
         pid);

/* Create client FIFO */
mkfifo(client_fifo, 0666);

printf("==== FIFO CLIENT ====\\n");
printf("Client PID: %d\\n", pid);

int server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1)
{
    perror("Server FIFO");
    unlink(client_fifo);
    return 1;
}

Request req;

req.client_pid = pid;

printf("Enter message: ");
fgets(req.message, sizeof(req.message), stdin);

req.message[strcspn(req.message, "\\n")] = '\\0';

write(server_fd, &req, sizeof(req));

close(server_fd);

/* Wait for server response */

int client_fd = open(client_fifo, O_RDONLY);

if (client_fd == -1)
{
    perror("Client FIFO");

```

```
        unlink(client_fifo);
        return 1;
    }

    char response[256];

    read(client_fd, response, sizeof(response));

    printf("Server response: %s\n", response);

    close(client_fd);

    unlink(client_fifo);

    return 0;
}
```

UBUNTU STEP-BY-STEP

STEP 1: Create the folder

```
mkdir os_practical6
cd os_practical6
```

STEP 2: Create server file

```
nano fifo_server.c
```

Paste the **server code** given above.

Save:

Ctrl + O

Enter

Ctrl + X

STEP 3: Create client file

```
nano fifo_client.c
```

Paste the **client code** given above.

Save:

Ctrl + O

Enter

Ctrl + X

STEP 4: Check the files

ls

Expected:

fifo_client.c

fifo_server.c

STEP 5: Compile server

gcc fifo_server.c -o fifo_server

STEP 6: Compile client

gcc fifo_client.c -o fifo_client

STEP 7: Check compiled files

ls

Expected:

fifo_client

fifo_client.c

fifo_server

fifo_server.c

STEP 8: START SERVER

Run:

```
./fifo_server
```

Expected:

```
===== FIFO SERVER =====  
Server is waiting for clients...
```

IMPORTANT

Leave this terminal running.

Do not press `Ctrl + C`.

STEP 9: OPEN SECOND TERMINAL

Open another Ubuntu terminal.

Type:

```
cd ~/os_practical6
```

Then:

```
./fifo_client
```

Expected:

```
===== FIFO CLIENT =====  
Client PID: 12345  
Enter message:
```

STEP 10: SEND MESSAGE

Type:

Hello Server

Press **Enter**.

Client output:

Server response: Server received your message: Hello Server

STEP 11: CHECK SERVER

Go back to the first terminal.

You should see:

Client 12345 connected
Message: Hello Server

STEP 12: TEST SECOND CLIENT

Open another terminal.

```
cd ~/os_practical6
```

Then:

```
./fifo_client
```

Enter:

Hello from Client 2

Expected:

Server response: Server received your message: Hello from Client 2

The server will show:

Client XXXXX connected
Message: Hello from Client 2

STEP 13: TEST THIRD CLIENT

Open another terminal.

```
cd ~/os_practical6
```

Run:

```
./fifo_client
```

Enter:

Hello from Client 3

Expected:

Server response: Server received your message: Hello from Client 3

This demonstrates **multiple clients communicating with the same server**.

OUTPUT

SERVER OUTPUT

```
===== FIFO SERVER =====
```

Server is waiting for clients...

Client 12345 connected

Message: Hello Server

Client 12346 connected

Message: Hello from Client 2

Client 12347 connected

Message: Hello from Client 3

CLIENT OUTPUT

```
===== FIFO CLIENT =====
```

Client PID: 12345

Enter message: Hello Server

Server response: Server received your message: Hello Server

RESULT

Thus, the **client-server application using Named Pipes (FIFOs)** was successfully implemented. The client successfully sent messages to the server, and the server processed and responded to the clients. Communication with multiple clients was also demonstrated.

PART – B

POSIX SIGNAL HANDLING

AIM

To create a **POSIX signal handling program** that captures **SIGINT**, **SIGTERM**, and **SIGUSR1**, demonstrates asynchronous event handling, and explains the role of signal handlers.

OBJECTIVES

1. To understand POSIX signals.
 2. To register signal handlers.
 3. To handle **SIGINT**.
 4. To handle **SIGTERM**.
 5. To handle **SIGUSR1**.
 6. To demonstrate asynchronous event handling.
 7. To understand the role of signal handlers.
-

REQUIREMENTS

- Ubuntu/Linux
- GCC compiler
- C programming language
- POSIX signal functions
- **signal()**
- **raise()**
- **kill()**

- `getpid()`
-

THEORY

A **signal** is a software notification sent to a process to inform it that a particular event has occurred.

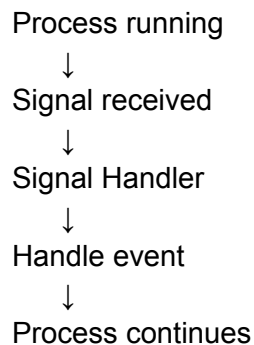
Signals can occur asynchronously, meaning the process does not have to be expecting the signal.

Signals used:

Signal	Meaning
<code>SIGINT</code> <code>T</code>	Interrupt signal, usually generated by <code>Ctrl+C</code>
<code>SIGTERM</code> <code>RM</code>	Request to terminate a process
<code>SIGUSR1</code> <code>R1</code>	User-defined signal 1

A **signal handler** is a function that is executed automatically when a particular signal is received.

Example:



ALGORITHM

1. Start the program.
 2. Get the process ID using `getpid()`.
 3. Register handlers for `SIGINT`, `SIGTERM`, and `SIGUSR1`.
 4. Display the process ID.
 5. Keep the process running.
 6. When a signal is received, execute the corresponding signal handler.
 7. Display the signal received.
 8. Continue execution for `SIGINT` and `SIGUSR1`.
 9. Terminate when `SIGTERM` is received.
 10. Stop the program.
-

PROGRAM

File Name

signal_handler.c

COPY-PASTE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received: Interrupt signal handled.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received: Terminating process.\n");
        exit(0);
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received: User-defined signal handled.\n");
    }
}
```

```
int main()
{
    printf("==== POSIX SIGNAL HANDLING =====\n");

    printf("Process ID: %d\n", getpid());

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Waiting for signals...\n");
    printf("Press Ctrl+C for SIGINT.\n");
    printf("Use kill -SIGUSR1 PID for SIGUSR1.\n");
    printf("Use kill -SIGTERM PID for SIGTERM.\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

UBUNTU STEP-BY-STEP

STEP 1: Create the file

Inside your folder:

nano signal_handler.c

Paste the complete code above.

Save:

Ctrl + O

Enter

Ctrl + X

STEP 2: Compile

```
gcc signal_handler.c -o signal_handler
```

STEP 3: Run

```
./signal_handler
```

You will get something similar to:

```
===== POSIX SIGNAL HANDLING =====  
Process ID: 12560  
Waiting for signals...  
Press Ctrl+C for SIGINT.  
Use kill -SIGUSR1 PID for SIGUSR1.  
Use kill -SIGTERM PID for SIGTERM.
```

Keep this terminal open.

STEP 4: TEST SIGUSR1

Open a second terminal.

Use the PID shown by your program.

For example, if the PID is:

```
12560
```

run:

```
kill -SIGUSR1 12560
```

The first terminal should display:

```
SIGUSR1 received: User-defined signal handled.
```

STEP 5: TEST SIGINT

Go back to the first terminal.

Press:

Ctrl + C

You should see:

SIGINT received: Interrupt signal handled.

The process **continues running** because our handler handles SIGINT instead of terminating the program.

STEP 6: TEST SIGTERM

Open/use the second terminal.

Run:

kill -SIGTERM 12560

Replace **12560** with **your actual PID**.

You should see:

SIGTERM received: Terminating process.

The program terminates.

EXPECTED OUTPUT

===== POSIX SIGNAL HANDLING =====

Process ID: 12560

Waiting for signals...

Press Ctrl+C for SIGINT.

Use kill -SIGUSR1 PID for SIGUSR1.

Use kill -SIGTERM PID for SIGTERM.

SIGUSR1 received: User-defined signal handled.

SIGINT received: Interrupt signal handled.


```
kruthi@LAPTOP-LP9QU3RR: ~  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_client  
==== FIFO CLIENT ====  
Client PID: 556  
Enter message: Hello server  
Server response: Server received your message: Hello server  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ cd ~/os_practical5/os_practical6  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ cd ~/os_practical5/os_practical6  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_client  
==== FIFO CLIENT ====  
Client PID: 566  
Enter message: Hello from Client 2  
Server response: Server received your message: Hello from Client 2  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_client  
==== FIFO CLIENT ====  
Client PID: 570  
Enter message: Hello from Client 3  
Server response: Server received your message: Hello from Client 3  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ |
```

```
===== FIFO CLIENT =====
Client PID: 570
Enter message: Hello from Client 3
Server response: Server received your message: Hello from Client
3
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ nano signal_handler.c
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ gcc signal_handler.c -o signal_handler
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./signal_handler
===== POSIX SIGNAL HANDLING =====
Process ID: 582
Waiting for signals...
Press Ctrl+C for SIGINT.
Use kill -SIGUSR1 PID for SIGUSR1.
Use kill -SIGTERM PID for SIGTERM.
^C
SIGINT received: Interrupt signal handled.
^C
SIGINT received: Interrupt signal handled.

^C
SIGINT received: Interrupt signal handled.
^C
SIGINT received: Interrupt signal handled.
^Z
[1]+  Stopped                  ./signal_handler
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ kill -SIGUSR1 582
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ |
```

```

kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ gcc fifo_client.c -o fifo_client
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ls
fifo_client fifo_client.c fifo_server fifo_server.c
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_server
==== FIFO SERVER ====
^C
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_server
==== FIFO SERVER ====
Server is waiting for clients...

Client 556 connected
Message: Hello server

Client 566 connected
Message: Hello from Client 2

Client 570 connected
Message: Hello from Client 3

```

```

==== POSIX SIGNAL HANDLING ====
Process ID: 595
Waiting for signals...
Press Ctrl+C for SIGINT.
Use kill -SIGUSR1 PID for SIGUSR1.
Use kill -SIGTERM PID for SIGTERM.

SIGUSR1 received: User-defined signal handled.

```

SIGUSR1 received: User-defined signal handled.

^C

SIGINT received: Interrupt signal handled.

^C

SIGINT received: Interrupt signal handled.

^C

SIGINT received: Interrupt signal handled.

^C

SIGINT received: Interrupt signal handled.

^C

SIGINT received: Interrupt signal handled.

SIGTERM received: Terminating process.

kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6\$ |

```
kruthi@LAPTOP-LP9QU3RR: ~  
SR1 582  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ fg  
./signal_handler  
  
SIGUSR1 received: User-defined signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^Z  
[1]+  Stopped                  ./signal_handler  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ls  
fifo_client    fifo_server    signal_handler  
fifo_client.c  fifo_server.c  signal_handler.c  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./signal_h  
andler  
===== POSIX SIGNAL HANDLING =====  
Process ID: 595  
Waiting for signals...  
Press Ctrl+C for SIGINT.  
Use kill -SIGUSR1 PID for SIGUSR1.  
Use kill -SIGTERM PID for SIGTERM.  
  
SIGUSR1 received: User-defined signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGINT received: Interrupt signal handled.  
^C  
SIGTERM received: Terminating process.  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ |
```

```
kruthi@LAPTOP-LP9QU3RR: ~  
kruthi@LAPTOP-LP9QU3RR:~$ cd ~/os_practical5/os_practical6  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ kill -SIGUSR1 595  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ^C  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ kill -SIGTERM 595  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ |
```

```
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ gcc fifo_client.c -o fifo_client  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ls  
fifo_client fifo_client.c fifo_server fifo_server.c  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_server  
==== FIFO SERVER ====  
^C  
kruthi@LAPTOP-LP9QU3RR:~/os_practical5/os_practical6$ ./fifo_server  
==== FIFO SERVER ====  
Server is waiting for clients...  
  
Client 556 connected  
Message: Hello server  
  
Client 566 connected  
Message: Hello from Client 2  
  
Client 570 connected  
Message: Hello from Client 3
```